Packet tracer

[switch/router configuration, VLAN, VLAN routing, L3 switch, Static Routing, Dynamic routing (RIP, OSPF)]

Creating a Simple Network (Networking Part)

Switch Configuration

Layer 2 and Layer 3 Switches

Layer 2 Switch
A Layer 2 switch uses MAC addresses to send data to the correct device, within the same network. However, it cannot send data between different networks; it only works within a single local network.

Layer 3 Switch
A Layer 3 switch uses both MAC addresses and IP addresses to manage data transfer, working similarly to a router. Because of this, it is able to route data between different networks, or even between different VLANs.

Manageable Switch vs Unmanageable Switch

(Correction: your notes had these two switched around - here is the correct version.)

Unmanageable Switch
With an unmanageable switch, we cannot change any settings at all. It works automatically as soon as we plug it in, with no configuration needed or possible.

Manageable Switch
With a manageable switch, we have full control over the network settings. We can change configurations to improve speed and security, and we can also do things like dividing the network into different VLANs.

Cable Connection Ports on a Switch

For example, a port labeled Fa0/0 refers to a Fast Ethernet port.

Fast Ethernet vs Gigabit Ethernet

Speed (the main difference):

- **Fast Ethernet:** Provides data transfer speeds of up to 100 Mbps (Megabits per second).
- **Gigabit Ethernet:** Provides much higher speeds, up to 1000 Mbps (also known as 1 Gbps).

Why do switches usually have many Fast Ethernet ports, but only 1 or 2 Gigabit Ethernet ports?

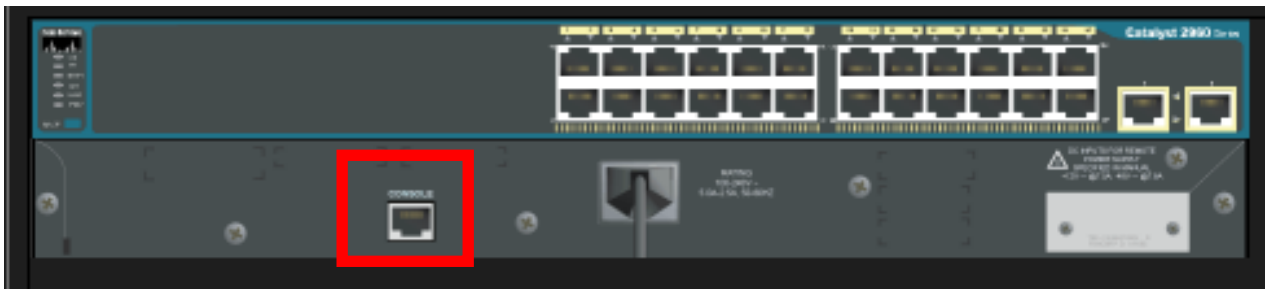
- **Uplink Connectivity:** The Gigabit ports are mainly used as "uplink ports," meaning they are used to connect the switch to a core switch, a high-speed server, or a router. Since these devices handle data coming from many different users all at once, they need a much wider "lane" (1000 Mbps) to avoid data traffic jams.

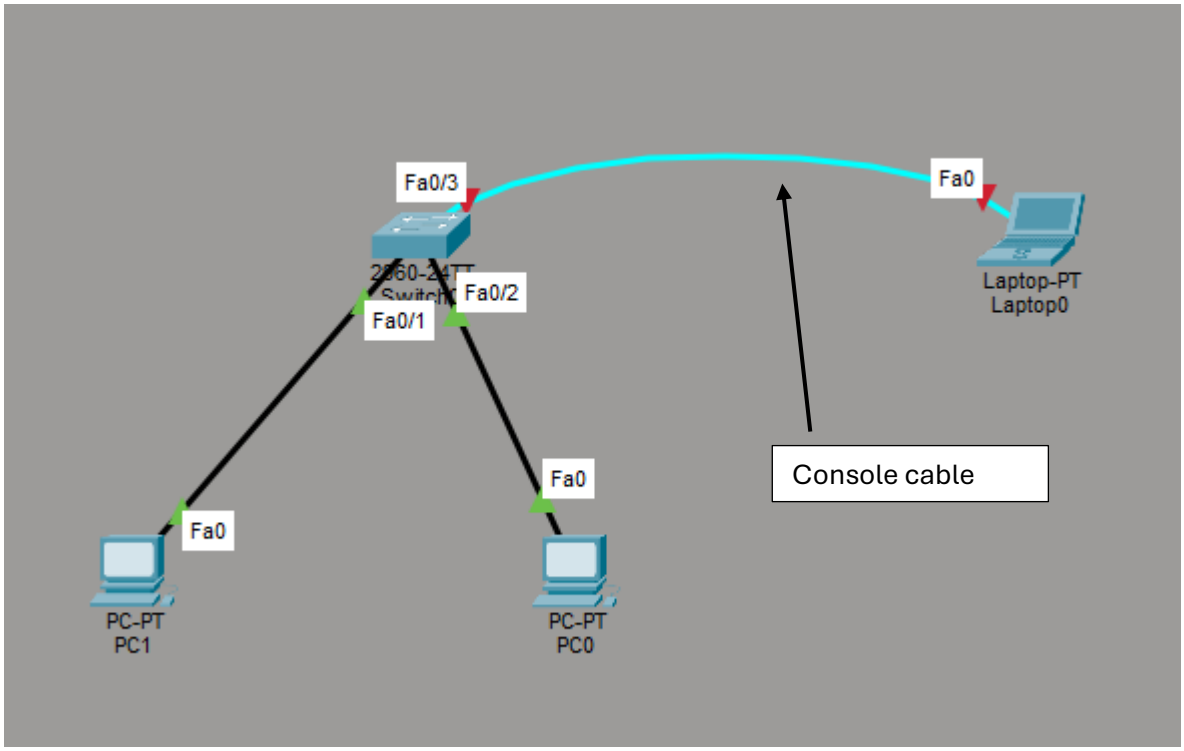
- **Cost-Effectiveness:** Making every single port a Gigabit port would increase the total manufacturing cost of the switch. For many everyday devices, like printers, VoIP phones, or basic office computers, a speed of 100 Mbps (Fast Ethernet) is already more than enough.

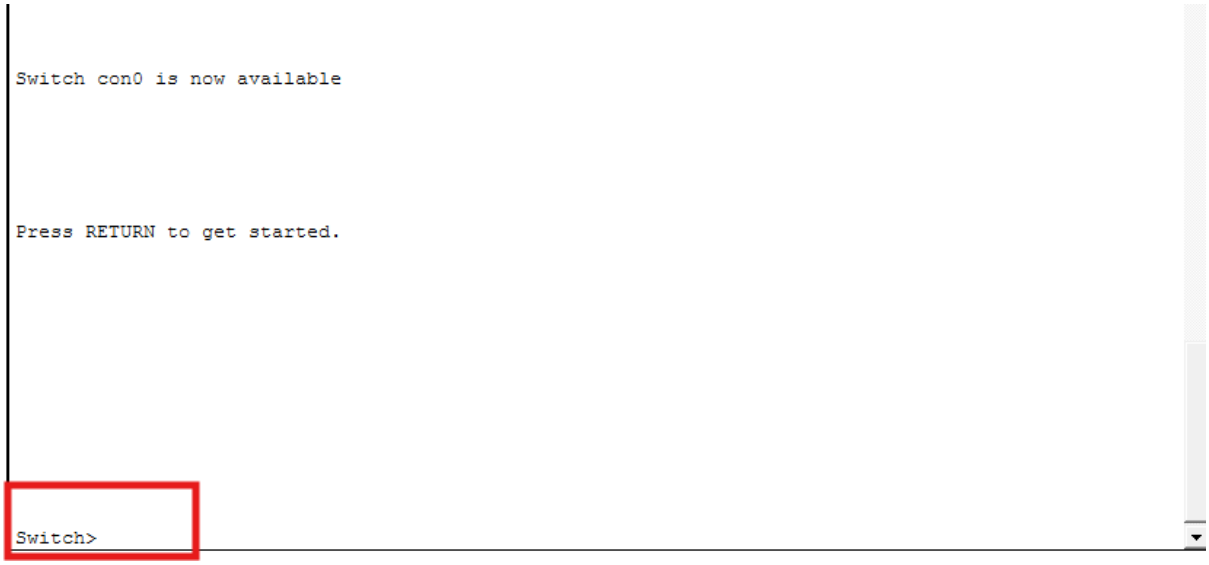
How to Configure a Switch Practically

To configure a switch, we usually use a Console Cable, which connects our laptop directly to the switch.

If the switch is brand new, or has been reset back to factory settings, it will not have any IP address assigned to it yet. In these cases, we must use a Console Cable for the very first login and initial configuration, since there is no other way to connect to it yet.







Unprivileged Mode (User EXEC Mode)

The symbol > represents the unprivileged mode of a switch, also known as User EXEC mode. In this mode, we cannot make any configuration changes to the switch.

However, even in this mode, we are still able to run a few basic commands, mostly for viewing information, not changing it.

How to See Available Commands in This Mode

To see the list of commands that are available to us in this mode, we simply type a question mark ? and press Enter.

For example:

Switch> ?

When we press Enter after typing ?, the switch will display a list of all the commands we are allowed to use at this current mode, along with a short description of what each command does. This is a helpful way to explore what is possible, without needing to memorize every command in advance.

```
Switch>
Switch>?
Exec commands:
  connect     Open a terminal connection
  disable     Turn off privileged commands
  disconnect  Disconnect an existing network connectio
  enable      Turn on privileged commands
  exit        Exit from the EXEC
  logout      Exit from the EXEC
  ping        Send echo messages
  resume      Resume an active network connection
  show        Show running system information
  ssh         Open a secure shell client connection
  telnet      Open a telnet connection
  terminal    Set terminal line parameters
  traceroute  Trace route to destination
Switch>
```

Show command

```
Switch>show ?
  arp                 Arp table
  cdp                 CDP information
  clock               Display the system clock
  crypto              Encryption module
  dtp                 DTP information
  etherchannel        EtherChannel information
  flash:              display information about flash: file system
  history             Display the session command history
  interfaces          Interface status and configuration
  ip                  IP information
  lldp                LLDP information
  mac                 MAC configuration
  mac-address-table   MAC forwarding table
  mls                 Show MultiLayer Switching information
  privilege           Show current privilege level
  sessions            Information about Telnet connections
  ssh                 Status of SSH server connections
  tcp                 Status of TCP connections
  terminal            Display terminal configuration parameters
  users               Display information about terminal lines
  version             System hardware and software status
  vlan                VTP VLAN status
  vtp                 VTP information
Switch>show
```

Important: Saving Switch Configurations

Once we make configuration changes on a switch, those changes are stored in the switch's running memory (RAM). However, if the switch loses power, for example, due to a power cut, or if it gets accidentally turned off, all of these configuration changes will be lost completely.

This happens because the running configuration is only stored temporarily in memory, and it is not automatically saved to permanent storage.

To prevent losing our work, it is very important to save the configuration after making any changes, so that it will still be there even after the switch restarts or loses power.

```
Switch#
Switch#copy running-config ?
  flash:           Copy to flash file
  ftp:             Copy to current system configuration
  scp:             Copy to scp: file system
  startup-config   Copy to startup configuration
  tftp:            Copy to current system configuration
Switch#copy running-config
```

Saving Configuration: The copy Command

To save our configuration on a switch, we use a command like this:

switch# copy running-config

The general syntax (structure) for this command is:

copy <source> <destination>

- **Source (running-config):** This refers to the configuration that is currently running in the switch's temporary memory (RAM). This is where all our recent changes are stored, but only temporarily.
- **Destination:** This is the location where we want to actually store the configuration file permanently, so that it does not disappear if the switch restarts or loses power.

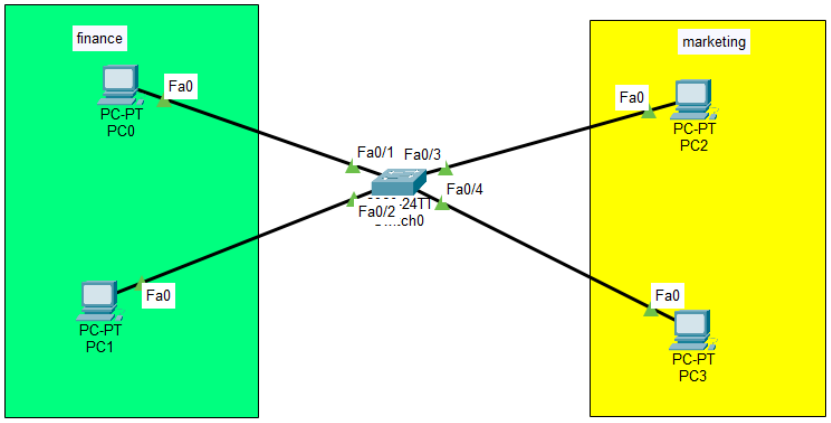
Common Destination Options

Depending on what we type after copy running-config, the configuration data will be saved to different locations:

Destination	Location	Purpose
startup-config	NVRAM	This is the most common and recommended choice. It saves our settings so that they will automatically load again, every time the switch restarts.
tftp	Remote Server	This sends the configuration to an external TFTP server, to be kept as a backup file.
flash	Flash Memory	This saves a copy of the configuration as a specific file, inside the switch's own local storage, similar to saving a text file onto a USB thumb drive.
ftp / scp	Secure Server	This transfers the configuration to a remote server, using more secure file transfer protocols.

In most everyday situations, we use copy running-config startup-config, since this is the simplest and most reliable way to make sure our settings are not lost when the switch restarts.

VLAN



create VLAN

SW1(config)#vlan 10
SW1(config-vlan)#name finance
SW1(config-vlan)#exit
SW1(config)#vlan 20
SW1(config-vlan)#name marketing
SW1(config-vlan)#exit

```
SW1(config)#vlan 10
SW1(config-vlan)#name finance
SW1(config-vlan)#exit
SW1(config)#vlan 20
SW1(config-vlan)#name marketing
SW1(config-vlan)#exit
SW1(config)#do sh vlan

VLAN Name                             Status    Ports
---- -------------------------------- --------- -------------------------------
1    default                          active    Fa0/1, Fa0/2, Fa0/3, Fa0/4
                                                Fa0/5, Fa0/6, Fa0/7, Fa0/8
                                                Fa0/9, Fa0/10, Fa0/11, Fa0/12
                                                Fa0/13, Fa0/14, Fa0/15, Fa0/16
                                                Fa0/17, Fa0/18, Fa0/19, Fa0/20
                                                Fa0/21, Fa0/22, Fa0/23, Fa0/24
                                                Gig0/1, Gig0/2
10   finance                          active
20   marketing                        active
1002 fddi-default                     active
1003 token-ring-default               active
1004 fddinet-default                  active
1005 trnet-default                    active

VLAN Type  SAID       MTU   Parent RingNo BridgeNo Stp  BrdgMode Trans1 Trans2
---- ----- ---------- ----- ------ ------ -------- ---- -------- ------ ------
1    enet  100001     1500  -      -      -        -    -        0      0
10   enet  100010     1500  -      -      -        -    -        0      0
20   enet  100020     1500  -      -      -        -    -        0      0
 --More--
```

How to Assign Ports to a VLAN

To assign a specific port on a switch to a VLAN, we need to follow a few steps.

Step 1: Access the Port

First, we need to enter the configuration mode for the specific port we want to assign. We do this using a command like:

int fa0/1

This command means we are now entering the interface (port) configuration mode, specifically for port Fa0/1 (Fast Ethernet port 1). Once we are inside this mode, any commands we type will only affect this particular port, not the whole switch.

Step 2: Select the Switchport Mode

After accessing the port, the next step is to select the switchport mode, which decides how this port will behave in relation to VLANs (for example, whether it belongs to just one VLAN, or whether it needs to carry traffic for multiple VLANs at once).

```
SW1(config)#int fa0/1
SW1(config-if)#sw
SW1(config-if)#switchport mo
SW1(config-if)#switchport mode ?
  access   Set trunking mode to ACCESS unconditionally
  dynamic  Set trunking mode to dynamically negotiate access or trunk mode
  trunk    Set trunking mode to TRUNK unconditionally
SW1(config-if)#switchport mode
```

Switchport Modes: Access Port vs Trunk Port

Access Port

An access port is usually assigned to only one single VLAN. This means any device connected to this port will only be part of that one specific VLAN, and nothing else.

We use access ports when the switch is transferring data to end devices, such as computers or printers, that belong to different VLANs, one device per port, with each port assigned to its own single VLAN.

Trunk Port

The main function of a trunk port is different, it carries data from multiple VLANs, all through the same single cable.

To understand when we use this, imagine we have two switches, and each of these switches has several different VLANs configured on it. If we want to connect these two switches together, and we want all their VLANs to be able to communicate properly across both switches, we would choose trunk mode for the port connecting them, instead of access mode.

This is done using a command like:

switchport trunk allowed vlan 10,20

This command tells the port to allow traffic from VLAN 10 and VLAN 20 (and any other VLANs we list) to pass through this single trunk connection.

Assigning a Port to a VLAN

Once we have decided whether a port should be an access port or a trunk port, we can then assign it to the desired VLAN, using the appropriate commands for that mode.

```
SW1(config)#int fa0/1
SW1(config-if)#sw
SW1(config-if)#switchport mo
SW1(config-if)#switchport mode ?
  access   Set trunking mode to ACCESS unconditionally
  dynamic  Set trunking mode to dynamically negotiate access or trunk mode
  trunk    Set trunking mode to TRUNK unconditionally
SW1(config-if)#switchport mode
```

Finally

```
VLAN Name                             Status    Ports
---- -------------------------------- --------- -------------------------------
1    default                          active    Fa0/2, Fa0/3, Fa0/4, Fa0/5
                                                Fa0/6, Fa0/7, Fa0/8, Fa0/9
                                                Fa0/10, Fa0/11, Fa0/12, Fa0/13
                                                Fa0/14, Fa0/15, Fa0/16, Fa0/17
                                                Fa0/18, Fa0/19, Fa0/20, Fa0/21
                                                Fa0/22, Fa0/23, Fa0/24, Gig0/1
                                                Gig0/2
10   finance                          active    Fa0/1
20   marketing                        active
1002 fddi-default                     active
1003 token-ring-default               active
1004 fddinet-default                  active
1005 trnet-default                    active
```

How to Assign Multiple Ports to a VLAN

Instead of configuring one port at a time, we can configure a whole range of ports together, using a single set of commands. Here is an example:

SW1(config)# int range fa0/1-10
SW1(config-if-range)# switchport mode access
SW1(config-if-range)# switchport access vlan 10
SW1(config-if-range)# no shut
SW1(config-if-range)# exit

```
SW1(config)#int range fa0/1-10
SW1(config-if-range)#switch
SW1(config-if-range)#switchport mode access
SW1(config-if-range)#switchport access vlan 10
SW1(config-if-range)#no shut
SW1(config-if-range)#exit
```

```
VLAN Name                             Status    Ports
---- -------------------------------- --------- -------------------------------
1    default                          active    Fa0/11, Fa0/12, Fa0/13, Fa0/14
                                                Fa0/15, Fa0/16, Fa0/17, Fa0/18
                                                Fa0/19, Fa0/20, Fa0/21, Fa0/22
                                                Fa0/23, Fa0/24, Gig0/1, Gig0/2
10   finance                          active    Fa0/1, Fa0/2, Fa0/3, Fa0/4
                                                Fa0/5, Fa0/6, Fa0/7, Fa0/8
                                                Fa0/9, Fa0/10
20   marketing                        active
1002 fddi-default                     active
1003 token-ring-default               active
1004 fddinet-default                  active
1005 trnet-default                    active
```

Let's break this down step by step:

- **int range fa0/1-10:** This command selects a whole range of ports at once, in this case, from Fa0/1 all the way to Fa0/10, so we don't need to configure each port one by one.
- **switchport mode access**: This sets all of these selected ports to access mode, meaning each one will belong to a single VLAN.
- **switchport access vlan 10:** This assigns all of these selected ports to VLAN 10.
- **no shut:** This command turns the ports on (enables them), since ports can sometimes be shut down (disabled) by default or by previous configuration.
- **exit:** This exits out of the current configuration mode.

After running these commands, if we check the VLAN list, we will see something like this:

VLAN	Name	Status	Ports
1	default	active	(remaining unassigned ports)
10	finance	active	Fa0/1 to Fa0/10
20	marketing	active	(its own assigned ports)

This confirms that ports Fa0/1 through Fa0/10 have now been successfully assigned to VLAN 10 (named "finance" in this example).

How Do Different VLANs Communicate With Each Other?

Now, let's say we have two separate VLANs, for example, VLAN 10 and VLAN 20. Even though they are on the same switch, devices in VLAN 10 cannot directly communicate with devices in VLAN 20, because VLANs are designed to act like completely separate networks, even when they share the same physical switch.

How can we make them communicate?

To allow communication between different VLANs, we need a router. This is because a router is specifically designed to connect and route data between different networks, and in this case, each VLAN is treated as its own separate network.

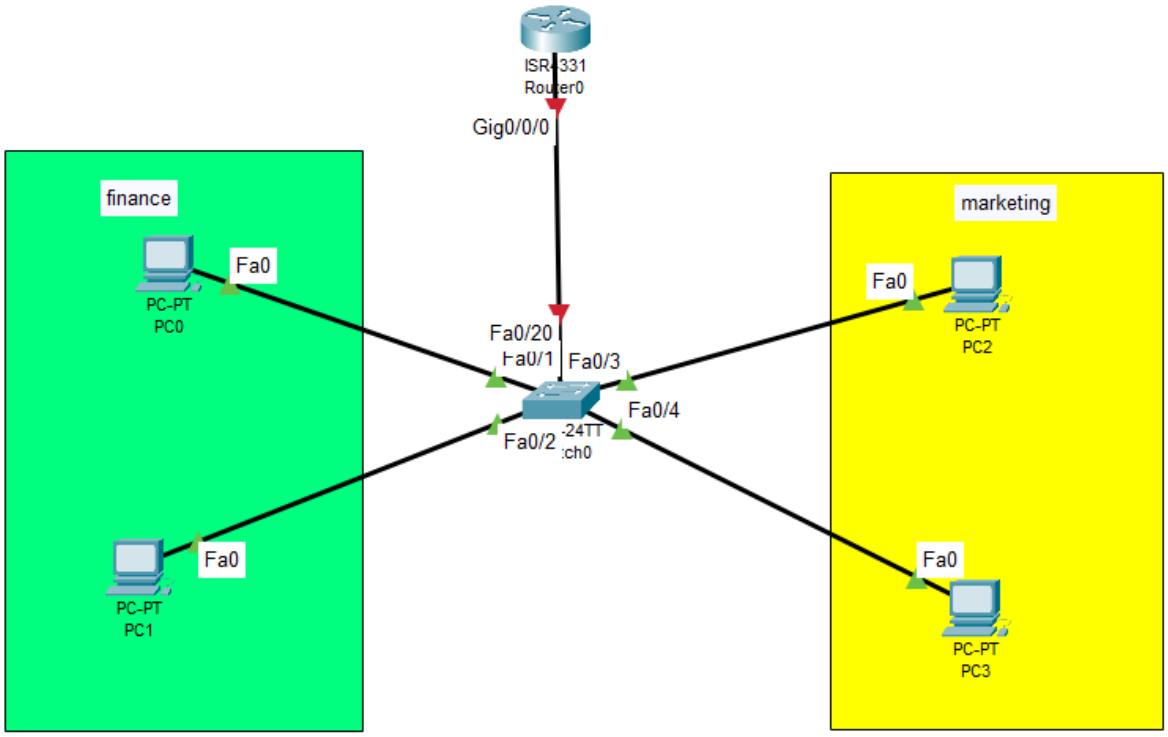
This process, where a router is used to allow communication between different VLANs, is called **Inter-VLAN Routing**.

Inter-VLAN Routing

Suppose we now have two different VLANs configured on the same switch: VLAN 10 and VLAN 20. Even though both VLANs exist on the same physical switch, devices inside VLAN 10 cannot directly talk to devices inside VLAN 20. This is because a VLAN is designed to act like its own separate, isolated network, even when it shares the same hardware with another VLAN.

So the question becomes: how can devices in different VLANs communicate with each other?

The answer is that we need a router. A router is specifically designed to connect and move data between different networks, and in this situation, each VLAN is treated as its own separate network. This entire process, where a router is used to allow communication between different VLANs, is called Inter-VLAN Routing.



In the topology above, we have a switch with two VLANs: the "finance" VLAN and the "marketing" VLAN. The switch is connected to a router through port Fa0/20, and this single cable will need to carry traffic for both VLANs at once.

Router Configuration (Sub-Interfaces)

On the router side, we need to configure the interface Gigabit Ethernet 0/0/0. However, since this single physical cable now needs to carry traffic for two different VLANs, we cannot simply configure the whole interface once. Instead, we divide this one physical interface into two logical sub-interfaces, one for each VLAN.

This is necessary because a single physical interface can only be assigned one IP address and one configuration. Since we have two separate VLANs (two separate networks) arriving over the same cable, we need two separate logical interfaces, each one dedicated to handling one VLAN's traffic.

```
Router(config)# interface gigabitEthernet 0/0/0.10
Router(config)# interface gigabitEthernet 0/0/0.20
```

Here, ".10" and ".20" are the sub-interface numbers we have chosen to represent VLAN 10 and VLAN 20. This numbering is just for our own reference and organization; the sub-interface number does not have to match the VLAN number, though it is common practice to keep them the same, so it is easier to remember which sub-interface belongs to which VLAN.

The encapsulation dot1Q Command

When a switch sends data out through a trunk port, it attaches a small VLAN ID label to each data frame, so that the receiving device knows exactly which VLAN that particular piece of data belongs to. This process of adding the VLAN label is called "tagging."

The problem is that a normal physical port on a router does not know how to read this VLAN label. A regular interface expects plain, untagged data, so if tagged data arrives, the router would not understand which VLAN it came from.

This is exactly why we created two sub-interfaces in the previous step. On each sub-interface, we use the encapsulation dot1Q command, to tell the router which specific VLAN tag that sub-interface should accept.

For example, when we type "encapsulation dot1Q 10" on the first sub-interface, we are telling the router: "On this sub-interface, only accept data packets that carry the VLAN 10 tag." Any data tagged with a different VLAN number will simply be ignored by this particular sub-interface, and will instead be picked up by the correct matching sub-interface.

A simple way to picture this is to imagine looking at the license plate of vehicles travelling down a single road (the physical cable), and then directing each vehicle to its correct destination, based only on what is written on its license plate (the VLAN ID).

```
R1(config)#int gigabitEthernet 0/0/0.10
R1(config-subif)#enc
R1(config-subif)#encapsulation do
R1(config-subif)#encapsulation dot1Q 10
R1(config-subif)#ip add
R1(config-subif)#ip address 192.168.10.254 255.255.255.0
```

In the example above, the router is configured so that sub-interface Gi0/0/0.10 is dedicated to VLAN 10, using the encapsulation dot1Q 10 command, and then an IP address is assigned to that sub-interface, which will act as the default gateway for all devices inside VLAN 10.

Configuring the Switch Port as a Trunk Port

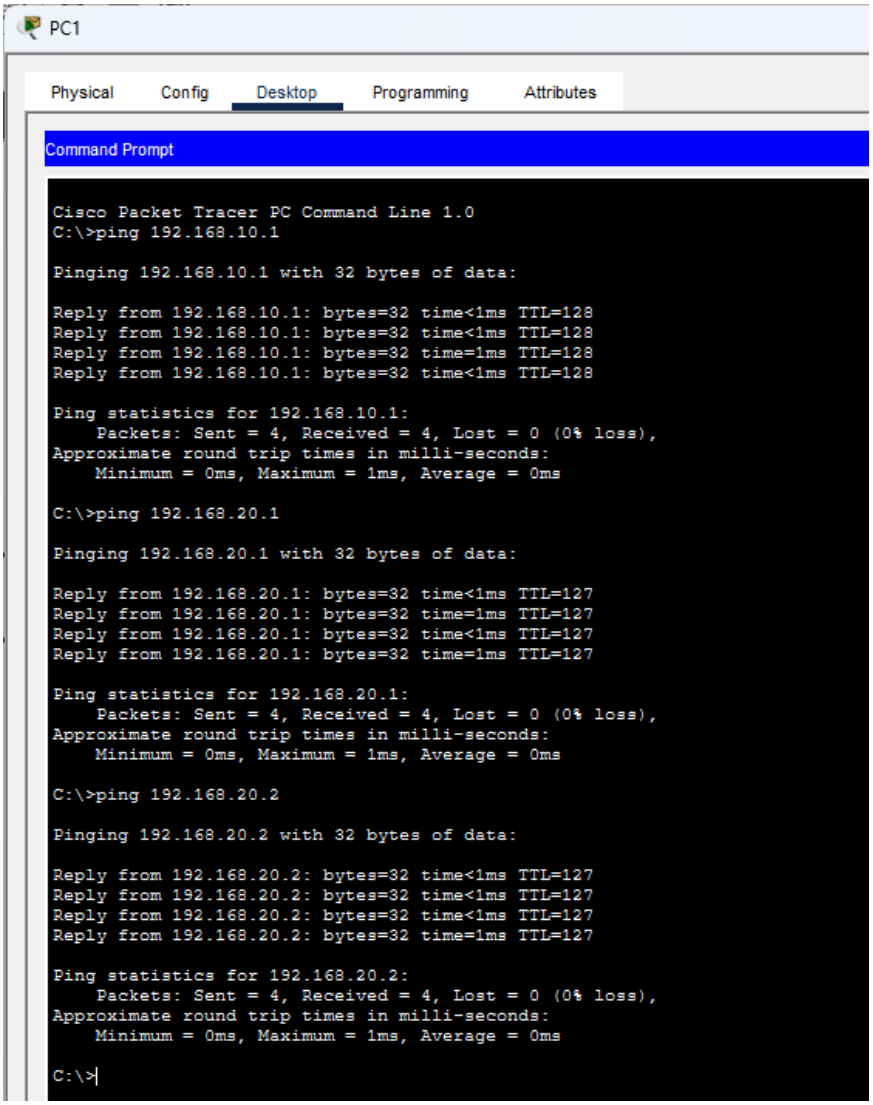
On the switch side, the port that connects to the router, in this case Fa0/20, must be configured as a trunk port. This is because this single port needs to carry traffic for multiple VLANs (VLAN 10 and VLAN 20) at the same time, over the same cable, going to the router.

```
SW1(config)#int fa0/20
SW1(config-if)#switch
SW1(config-if)#switchport mo
SW1(config-if)#switchport mode ?
  access   Set trunking mode to ACCESS unconditionally
  dynamic  Set trunking mode to dynamically negotiate access or trunk mode
  trunk    Set trunking mode to TRUNK unconditionally
SW1(config-if)#switchport mode trunk
SW1(config-if)#swit
SW1(config-if)#switchport tr
SW1(config-if)#switchport trunk?
trunk
SW1(config-if)#switchport trunk all
SW1(config-if)#switchport trunk allowed ?
  vlan  Set allowed VLANs when interface is in trunking mode
SW1(config-if)#switchport trunk allowed vlan?
vlan
SW1(config-if)#switchport trunk allowed vlan 10,20
SW1(config-if)#
SW1(config-if)#
SW1(config-if)#
SW1(config-if)#
SW1(config-if)#
SW1(config-if)#
SW1(config-if)#
SW1(config-if)#no shut
SW1(config-if)#
SW1(config-if)#
SW1(config-if)#
SW1(config-if)#exit
```

In this configuration, we first enter the interface, then set the switchport mode to trunk using "switchport mode trunk". After that, we specify which VLANs are allowed to pass through this trunk, using the command "switchport trunk allowed vlan 10,20". Finally, we make sure the port is active using "no shut", and then exit the configuration.

Result: Testing the Configuration

After completing the router and switch configuration, we can test whether devices in different VLANs are able to communicate with each other, by pinging between devices in VLAN 10 and VLAN 20.

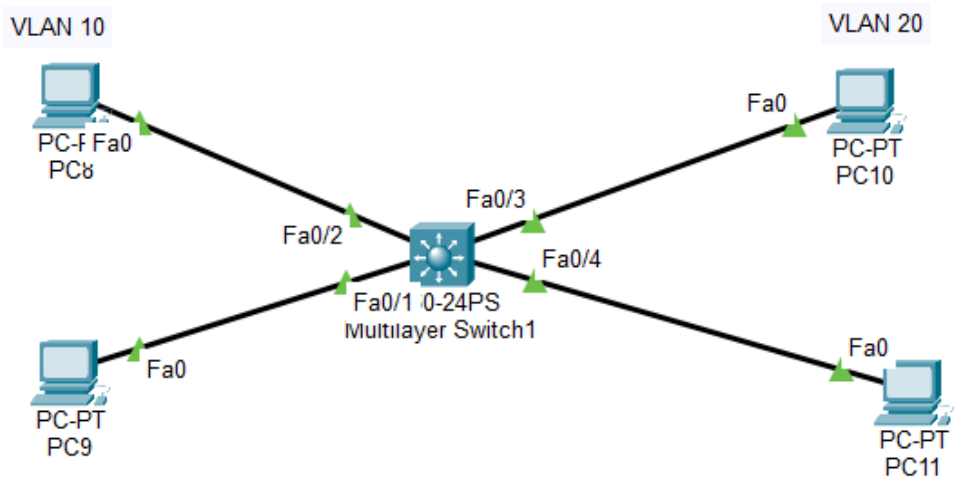


As we can see in the result above, PC1 is successfully able to ping devices with addresses in both the 192.168.10.x range (VLAN 10) and the 192.168.20.x range (VLAN 20), with zero packet loss. This confirms that the router is now successfully routing traffic between the two VLANs, and Inter-VLAN Routing is working correctly.

Inter-VLAN Communication Without a Router (Using a Layer 3 Switch)

In the previous method, we used a separate physical router to allow communication between VLANs. However, there is another way to achieve the same result, without needing a separate router at all: by using a Layer 3 switch instead.

Unlike a normal Layer 2 switch, a Layer 3 switch is capable of understanding IP addresses, and can perform routing directly within the switch itself. Because of this ability, a Layer 3 switch can handle Inter-VLAN routing on its own, without needing to send traffic out to an external router.



In this topology, we again have two VLANs, VLAN 10 and VLAN 20, but this time they are both connected to a Layer 3 switch, which will handle the routing between them directly.

```
Switch(config)#ip ?
  access-list       Named access-list
  arp               IP ARP global configuration
  cef               Cisco Express Forwarding
  default-gateway   Specify default gateway (if not routing IP)
  default-network   Flags networks as candidates for default routes
  dhcp              Configure DHCP server and relay parameters
  domain            IP DNS Resolver
  domain-lookup     Enable IP Domain Name System hostname translation
  domain-name       Define the default domain name
  forward-protocol  Controls forwarding of physical and directed IP broadcasts
  ftp               FTP configuration commands
  host              Add an entry to the ip hostname table
  local             Specify local options
  name-server       Specify address of name server to use
  nat               NAT configuration commands
  route             Establish static routes
  routing           Enable IP routing
  scp               Scp commands
  ssh               Configure ssh options
  tcp               Global TCP parameters
```

The image above shows the available options under the "ip ?" command on the switch. Notice the "routing" option, highlighted with the arrow. This is the specific command we need to enable, so that the switch is allowed to perform IP routing, in addition to its normal Layer 2 switching functions.

Step 1: Assign Ports to VLANs

Just like before, the first step is to create the VLANs, and then assign the physical switch ports to their correct VLAN, using access mode.

```
Switch#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
Switch(config)#
Switch(config)#
Switch(config)#vlan 10
Switch(config-vlan)#exit
Switch(config)#vlan 20
Switch(config-vlan)#exit
Switch(config)#int range fa0/1-2
Switch(config-if-range)#swithport access vlan 10
                               ^
% Invalid input detected at '^' marker.

Switch(config-if-range)#switchport access vlan 10
Switch(config-if-range)#exit
Switch(config)#int range fa0/3-4
Switch(config-if-range)#switchport access vlan 20
Switch(config-if-range)#exit
```

In the example above, VLAN 10 and VLAN 20 are created first. Then, ports Fa0/1 and Fa0/2 are assigned to VLAN 10, while ports Fa0/3 and Fa0/4 are assigned to VLAN 20, using the "switchport access vlan" command on each port range.

Step 2: Add a Gateway (SVI) to Each VLAN

Next, we need to give each VLAN its own gateway address, directly on the switch. This is done by creating something called a Switch Virtual Interface (SVI) for each VLAN. An SVI is a virtual interface that represents the VLAN itself, and it is given an IP address, which then acts as the default gateway for all devices inside that VLAN.

```
Switch(config)#int
Switch(config)#interface vlan 10
Switch(config-if)#
%LINK-5-CHANGED: Interface Vlan10, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface Vlan10, changed state to up

Switch(config-if)#
Switch(config-if)#ip add 192.168.10.254 255.255.255.0
Switch(config-if)#exit
Switch(config)#
Switch(config)#int
Switch(config)#interface vlan 20
Switch(config-if)#
%LINK-5-CHANGED: Interface Vlan20, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface Vlan20, changed state to up

Switch(config-if)#ip add 192.168.20.254 255.255.255.0
Switch(config-if)#
Switch(config-if)#exit
```

In the example above, we enter interface vlan 10, and assign it the IP address 192.168.10.254, which will act as the gateway address for all devices in VLAN 10. We then repeat the same process for VLAN 20, assigning it the gateway address 192.168.20.254. Each device inside a VLAN should have its default gateway set to match its VLAN's SVI address.

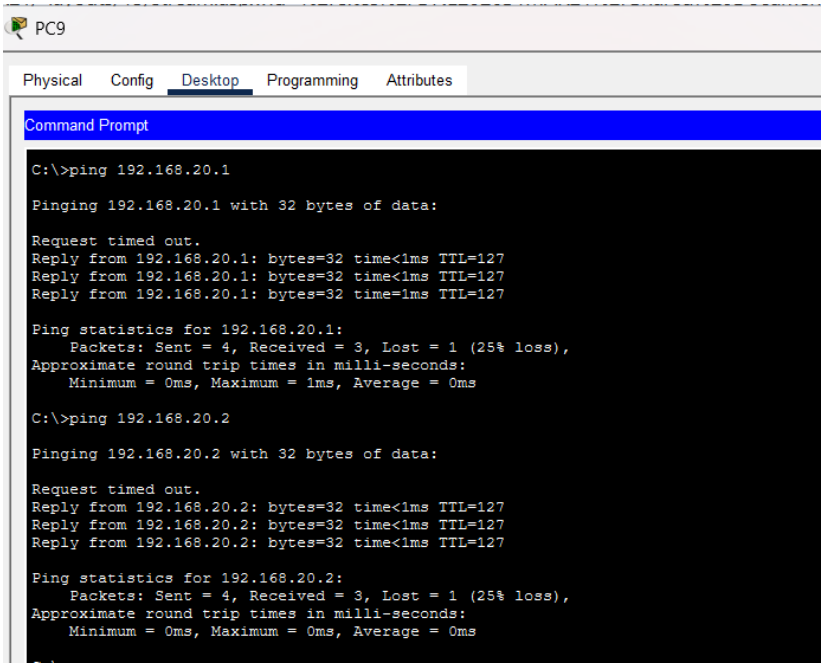
Step 3: Enable IP Routing

Even after creating the gateway addresses for each VLAN, the switch will still not actually route traffic between the VLANs, until we explicitly turn on IP routing. Without this step, the switch will only behave like a normal Layer 2 switch, even though it has Layer 3 capability.

```
Switch(config)#
Switch(config)#
Switch(config)#
Switch(config)#
Switch(config)#
Switch(config)#ip routing
Switch(config)#
Switch(config)#
Switch(config)#
```

The command "ip routing" enables full Layer 3 routing functionality on the switch. Once this is turned on, the switch will now be able to move traffic between VLAN 10 and VLAN 20, using the gateway addresses we configured in the previous step.

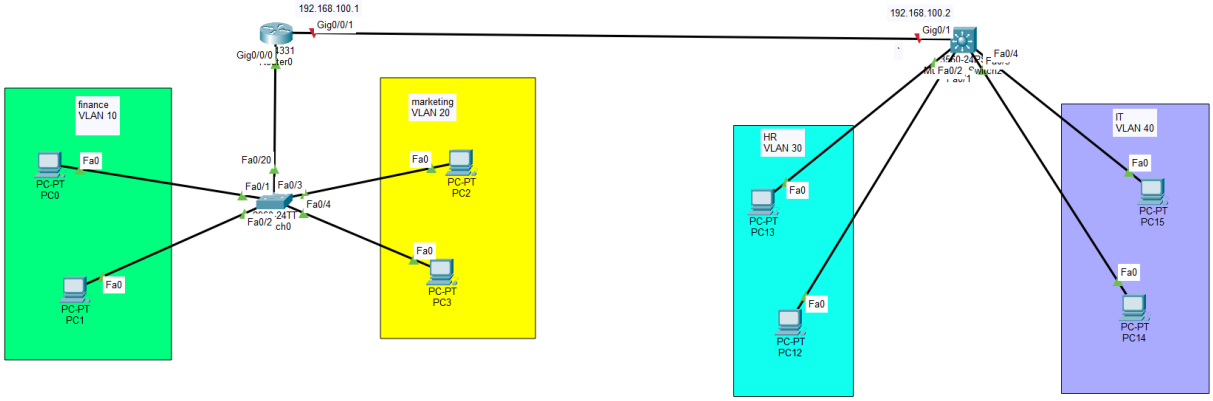
Result: Testing the Layer 3 Switch Configuration



In this result, we can see that PC9 is able to successfully ping addresses in the 192.168.20.x range, confirming that devices in different VLANs can now communicate with each other, purely through the Layer 3 switch, without needing any separate physical router.

Connecting a Router with a Layer 3 Switch

Sometimes, a network may use both a router and a Layer 3 switch together, especially in larger networks with many VLANs spread across different areas. In this situation, we can also use a Layer 3 switch to connect with a router, but this cannot be done using a normal Layer 2 switch, since a Layer 2 switch has no understanding of IP addressing.



In the topology above, notice that the router and the Layer 3 switch (labeled Switch2) are connected to each other directly, using their own IP addresses (192.168.100.1 on the router's side, and 192.168.100.2 on the switch's side). This connection allows traffic from VLANs on one side of the network to reach VLANs and devices on the other side.

How do we make this connection work? We can achieve this using static routing. To begin, we first need to assign IP addresses directly to the physical ports connecting the router and the switch (in this example, the router's Gig0/0/1 port, and the switch's Gig0/1 port).

The no switchport Command

When we want to assign an IP address directly to a port on a Layer 3 switch, we need to first issue a special command on that port, called "no switchport".

```
MSW1(config)#int gigabitEthernet 0/1
MSW1(config-if)#no sw
MSW1(config-if)#no switchport
MSW1(config-if)#ip add 192.168.100.2 255.255.255.0
```

When we issue the "no switchport" command, all of the normal Layer 2 features of that port, such as VLAN membership and MAC address learning, are removed from that port. This is necessary because, technically, it is not possible to directly assign an IP address to a Layer 2 port, since IP addressing is a Layer 3 (Network Layer) function, not a Layer 2 function.

Once we run this command, that particular port stops behaving like a normal switch port, and instead starts behaving like a router port, which means it can now be assigned its own IP address, just like a router interface would be.

Static Routing

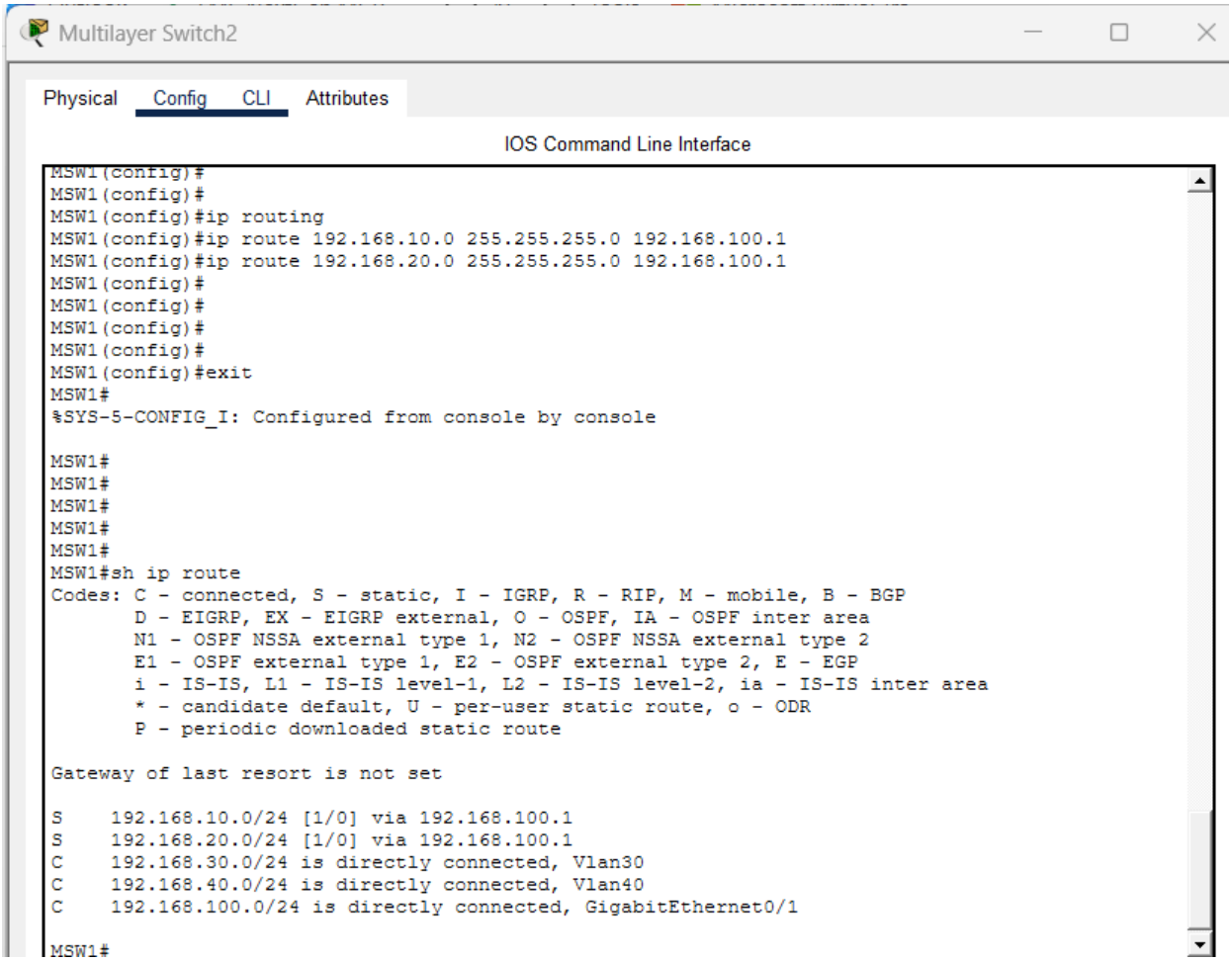
Static routing is a method where we manually tell a router or Layer 3 switch exactly how to reach a specific destination network, by directly typing in the route ourselves. This is different from dynamic routing, where the routing information is exchanged and learned automatically.

The general command structure for static routing is:

```
ip route [Destination_Network] [Subnet_Mask] [Next_Hop_IP]
```

This means: to reach this specific destination network, using this specific subnet mask, send the traffic to this particular next-hop IP address.

On the Layer 3 Switch



In the example above, on the switch (MSW1), we can see the commands "ip route 192.168.10.0 255.255.255.0 192.168.100.1" and "ip route 192.168.20.0 255.255.255.0 192.168.100.1" being used. These commands tell the switch: "To reach the 192.168.10.0 network and the 192.168.20.0 network, send the traffic to 192.168.100.1," which is the router's IP address. The routing table shown afterward (using the "sh ip route" command) confirms these static routes are now active, marked with an "S" for static.

On the Router

```
R1(config)#ip route 192.168.30.0 255.255.255.0 192.168.100.2
R1(config)#ip route 192.168.40.0 255.255.255.0 192.168.100.2
```

On the router's side, we do the same thing in reverse, telling the router how to reach the networks that exist on the switch's side. Here, the commands "ip route 192.168.30.0 255.255.255.0 192.168.100.2" and "ip route 192.168.40.0 255.255.255.0 192.168.100.2" tell the router: "To reach the 192.168.30.0 and 192.168.40.0 networks, send the traffic to 192.168.100.2," which is the switch's IP address.

By configuring static routes on both sides like this, in both directions, we make sure that traffic can successfully flow from any VLAN on one side of the network, all the way to any VLAN on the other side, and back again.

Default Routing

A default route is a special, simplified type of static route. Instead of manually writing out a separate route for every single possible destination network, a default route acts as a catch-all, saying: "If you don't know where else to send this traffic, send it here."

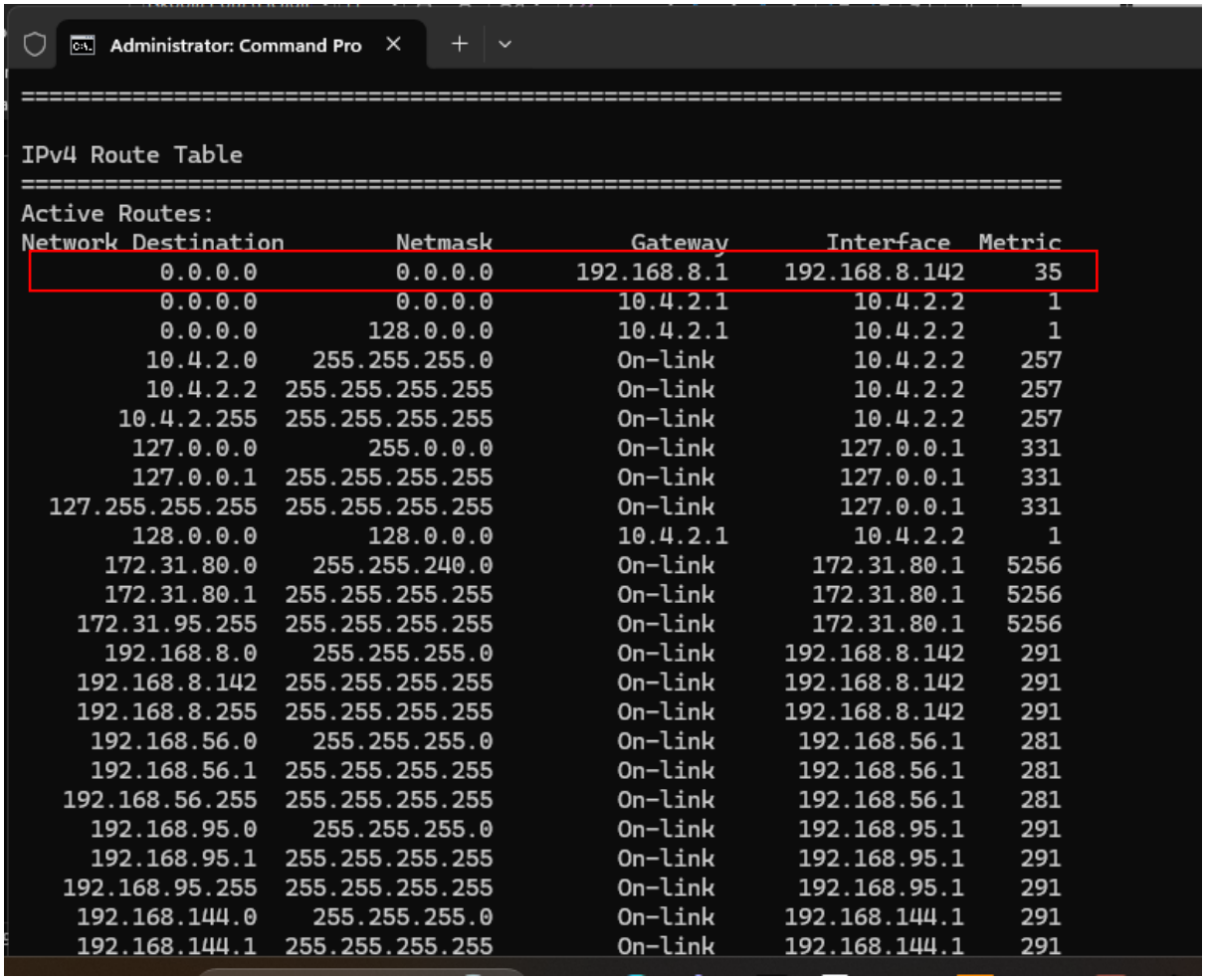
```
ip route 0.0.0.0 0.0.0.0 192.168.100.1
ip route 0.0.0.0 0.0.0.0 192.168.100.2
```

Here, "0.0.0.0 0.0.0.0" represents "any destination network," meaning this single route will match any IP address that doesn't have a more specific matching route already defined. Once a default route like this is configured, all traffic to unknown or unlisted networks will automatically be sent to that specified next-hop address, allowing any IP address to be reached, without needing to write a separate static route for every single network individually.

Checking Static Routes on Your Own Computer

Static routing is not only something we configure on routers and switches, our own everyday computers also keep their own local routing table, which decides where to send outgoing traffic.

We can view this routing table ourselves. To do this, we open Command Prompt as an administrator, and then type the command "**route print**". This will display our computer's own IPv4 and IPv6 routing tables, showing us exactly which gateway our computer uses for different destination networks.



In the result above, we can see various routes listed. The first line, with destination 0.0.0.0, represents our computer's default gateway, meaning that any traffic not matching a more specific rule will be sent through that gateway address. This is very similar in concept to the default routing we set up on the router earlier, showing that this same fundamental idea of static routing applies to ordinary computers as well.

Dynamic Routing

Dynamic routing is a completely different approach to routing, compared to static routing. Instead of manually typing in every single route ourselves, dynamic routing allows a router to automatically exchange routing information with other routers, and independently figure out all the available paths across the network on its own.

This means that, unlike static routing, we do not need to manually write out each individual path. Instead, we simply tell the router which networks it is directly connected to, and the dynamic routing protocol takes care of sharing that information with other routers, and calculating the best paths automatically.

There are several common dynamic routing protocols, each with its own way of calculating the best path:

- RIP - Routing Information Protocol (used mainly for small networks)
- OSPF - Open Shortest Path First

- EIGRP - Enhanced Interior Gateway Routing Protocol
- BGP - Border Gateway Protocol

RIP Protocol

RIP, which stands for Routing Information Protocol, is one of the oldest and simplest dynamic routing protocols in existence. Because of its simplicity, it is mainly used for small networks, and is not well suited for very large, complex networks.

RIP works based on something called Distance Vector technology. In this context, the word "distance" refers to something called the Hop Count.

Hop Count simply refers to the number of routers that a data packet must pass through, in order to reach its final destination. RIP always chooses the path with the lowest hop count, meaning the path that passes through the fewest number of routers, regardless of how fast or slow each individual connection actually is.

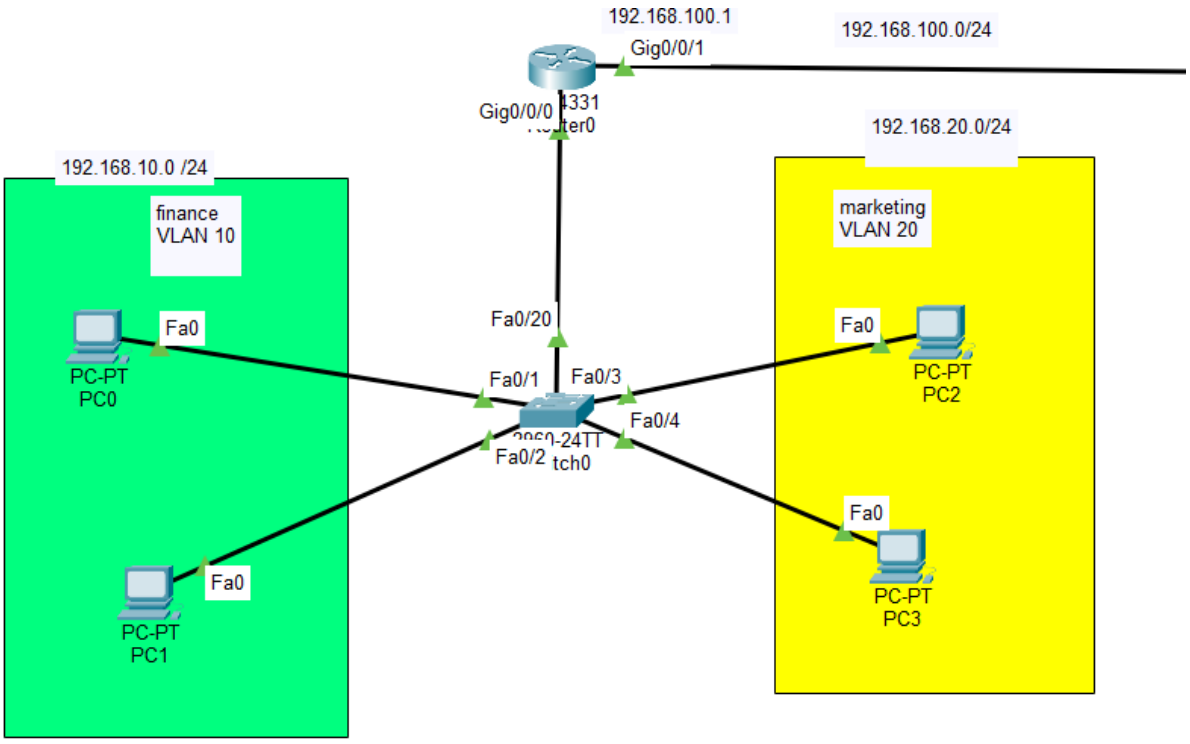
RIP does have a limitation: it supports a maximum hop count of only 15 routers. If a destination is 16 hops away or further, RIP considers that destination completely unreachable.

There are two versions of RIP that are commonly discussed:

- RIP Version 1: A very old version. It uses classful routing, meaning it does not support subnetting.
- RIP Version 2: The version currently used today. It supports classless routing (VLSM and subnetting), and is generally more secure than version 1.

When configuring RIP on a router, all we need to do is tell the router which networks are directly connected to it; RIP will handle sharing that information with other routers automatically.

```
Router(config)# router rip
Router(config-router)# version 2
Router(config-router)# network 192.168.10.0
Router(config-router)# network 192.168.20.0
Router(config-router)# network 192.168.100.0
Router(config-router)# no auto-summary
```



Why Use "no auto-summary"?

The "no auto-summary" command is used to make sure RIP correctly and accurately identifies each individual subnetted network separately, such as the Finance network (192.168.10.0) and the Marketing network (192.168.20.0), without automatically merging or summarizing them together into one broader network range, which could cause confusion or incorrect routing behavior.

```
MSW1(config)#router rip
MSW1(config-router)#version 2
MSW1(config-router)#no auto
MSW1(config-router)#no auto-summary
MSW1(config-router)#network 192.168.30.0
MSW1(config-router)#network 192.168.40.0
MSW1(config-router)#network 192.168.100.0
MSW1(config-router)#exit
```

```
Codes: C - connected, S - static, I - IGRP, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

R    192.168.10.0/24 [120/1] via 192.168.100.1, 00:00:01, GigabitEthernet0/1
R    192.168.20.0/24 [120/1] via 192.168.100.1, 00:00:01, GigabitEthernet0/1
C    192.168.30.0/24 is directly connected, Vlan30
C    192.168.40.0/24 is directly connected, Vlan40
C    192.168.100.0/24 is directly connected, GigabitEthernet0/1
```

In the routing table shown above, we can see routes marked with an "R", which indicates that these particular routes were learned dynamically, through the RIP protocol, rather than being manually typed in as static routes.

OSPF Protocol

Unlike RIP, OSPF (Open Shortest Path First) works based on a different technology, called Link State technology. In this method, the router does not simply look at the hop count (the number of routers along the way), like RIP does.

Instead, OSPF chooses the best path based on something called "Cost". Generally speaking, the higher the speed (bandwidth) of a particular connection, the lower its associated cost value will be. OSPF will always choose the path with the overall lowest total cost, which usually results in choosing the fastest available path, rather than simply the path with the fewest hops.

One of the special abilities of OSPF is that it allows us to divide and organize a large network into smaller, more manageable parts, called Areas.

Every OSPF network must have an Area 0, also known as the Backbone Area. All other areas in the network must connect back to this central backbone area, either directly or indirectly.

The general command structure used to configure OSPF is:

```
network [Network_Address] [Wildcard_Mask] area [Area_ID]
```

Here, the wildcard mask is essentially the reverse of a normal subnet mask; wherever the subnet mask has a 0, the wildcard mask will have a 1, and wherever the subnet mask has a 1, the wildcard mask will have a 0.

Configuring OSPF on the Router

```
Router(config)# router ospf 1
Router(config-router)# router-id 1.1.1.1
Router(config-router)# network 192.168.10.0 0.0.0.255 area 0
Router(config-router)# network 192.168.20.0 0.0.0.255 area 0
Router(config-router)# network 192.168.100.0 0.0.0.255 area 0
```

Configuring OSPF on the Layer 3 Switch

```
Switch2(config)# ip routing
Switch2(config)# router ospf 1
Switch2(config-router)# router-id 2.2.2.2
Switch2(config-router)# network 192.168.30.0 0.0.0.255 area 0
Switch2(config-router)# network 192.168.40.0 0.0.0.255 area 0
Switch2(config-router)# network 192.168.100.0 0.0.0.255 area 0
```

Notice that each device is given its own unique router-id (1.1.1.1 for the router, and 2.2.2.2 for the switch). This ID is used by OSPF to uniquely identify each device participating in the OSPF process.

Checking OSPF Neighbors

Once OSPF is configured on both devices, we can check whether they have successfully recognized each other as OSPF neighbors, using the command "show ip ospf neighbor".

```
R1#sh ip ospf n
R1#sh ip ospf neighbor


Neighbor ID     Pri   State           Dead Time   Address         Interface
1.1.1.1           1   FULL/DR         00:00:39    192.168.100.2   GigabitEthernet0/0/1
R1#
```

In the result above, we can see that the router has successfully identified a neighbor with the ID 1.1.1.1, in a "FULL/DR" state, which confirms that the OSPF relationship between the two devices has been formed successfully, and they are now able to exchange routing information dynamically with each other.

Important Note: Access Control (ACL)

Access Control Lists (ACL) are used to control and filter what type of traffic is allowed or denied within a network. This is a separate, important topic on its own. A shared reference link with further material on how to create access control lists, and how they work, has been provided separately alongside this document.